

Chapter 4_Evaluate AI system

An application that is deployed but cannot be evaluated is the worst.

Evaluation Criterion

Evaluation-driven Development = Test-driven Development

- Defining the evaluation criteria before building.
- Being able to build reliable evaluation pipelines will unlock so many new applications
- In an AI application, therefore, should start with a list of evaluation criteria specific to the application. Criteria:
 - **Domain-specific capability:**
 - For example, to build a coding agent, you need a model that can write code.
 - **Generation capability:**
 - For example, in the text generation task,
 - NLG
 - We have to measure the coherence and the fluency
 - **Coherence** = how well-structured the whole text is (does it follow a logical structure)
 - **Fluency** = measures whether the text is grammatically correct and natural-sounding
 - **Factual Consistency**
 - Hallucination can sometimes be helpful in some creative tasks, but not in tasks that depend on factuality.
 - The factual consistency is much easier to verify against the explicit fact.
 - The factual consistency can be verified under two main settings:
 - **Local factual consistency:** The output is evaluated against a context. The output is considered factually consistent if it is supported by the given context. For example, if the output says "the sky is blue," and the given context is "the sky is purple," then this output is factually consistent.
 - It is important for tasks with limited scopes, such as summarization (the summary should be consistent with the original statement), customer support agent (the chatbot has to be consistent with the store's policy), and business analysis (the extracted insight should be consistent with the data)
 - **Global factual consistency:** The output is evaluated against open knowledge. If the model outputs "the sky is blue" and it is

a commonly accepted fact that the sky is blue, this statement is considered factually correct.

- It is important for tasks with broad scopes, such as general chatbots, fact-checking, market research, etc

Bottleneck of the factual consistency: determining what the facts are

- AI as a judge in the factual consistency:
 - **Self-verification:** Given response R, the SelfcheckAI generates the new response N, then it measures how consistent R is with respect to N. This approach is practical but considerably expensive in both cost and latency, as it requires the AI to generate and evaluate the response.
 - Knowledge-augmented verification: SAFE (Search Augmented Factuality Evaluator, introduced by Google DeepMind), published in the paper "Long-Form Factuality in Large Language Models," works by leveraging search engine results to verify the response. It works in the following four steps:
 - Use an AI model to decompose the response into individual statements
 - Revise each statement to make it self-contained. For example, the "it" in the statement "it opened in the 20th century" should be changed to the original subject
 - For each statement, propose fact-checking queries to send to a Google Search API
 - Use AI to determine whether the statement is consistent with the research results.

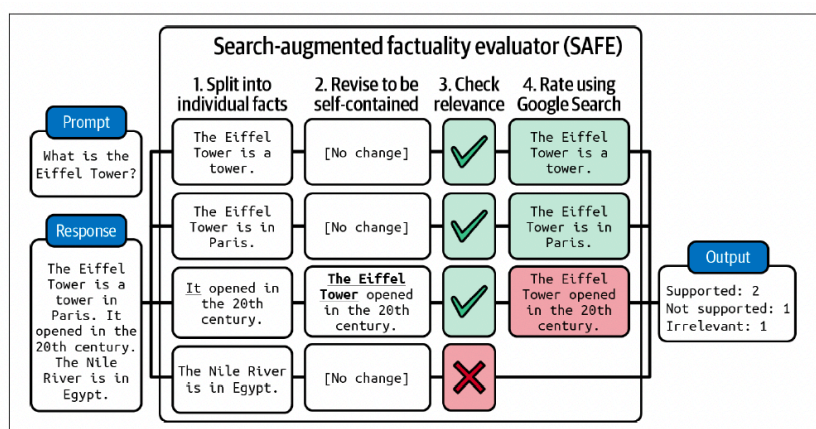


Figure 4-1. SAFE breaks an output into individual facts and then uses a search engine to verify each fact. Image adapted from Wei et al. (2024).

- Verifying whether a statement is consistent with a given context can also be framed as a textual entailment task, which is a long-standing NLP task. **Textual entailment** = is the task of determining the relationship between two statements.

- **Entailment:** The hypothesis can be inferred from the premise. For example: Mary likes apples.
 - **Contradiction:** The hypothesis contradicts the premise. For example: Mary hates oranges.
 - **Neutral:** the premise neither entails nor contradicts the hypothesis. For example: Mary likes chickens.
- Factual consistency is a very crucial evaluation criterion for an RAG system. Given a query, a RAG system retrieves relevant information from external databases to supplement the model's context. So the generated response should be factually consistent with the retrieved context.
- Other than the factual consistency, there are many ways in which a model's outputs can be harmful. Unsafe contents might belong to one of the following categories:
 1. Inappropriate language, including profanity and explicit content
 2. harmful recommendations and tutorials, such as "step-by-step guide to rob a bank" or encouraging users to engage in self-destructive behavior
 3. Hate speech, including racist, sexist, homophobic speech, and other discriminatory behavior.
 4. Violence, including threats and graphic details
 5. Stereotypes, such as always using female names for nurses and male names for CEOs.
 6. Bias toward a political or religious ideology
 - **Instruction following capability:** "How good is this model at following the instructions you give it?" If the model is bad at following the instructions, it does not matter how good your instructions are; the output will be bad.
 - **Instruction following capability** is crucial for applications that require structured outputs, such as in JSON format or matching a regular expression (regex)
 - For example, you ask the model to classify an input as A, B, or C, but the model outputs "that's correct"; this output is not very helpful and will likely break downstream applications that expect only A, B, or C.
 - "How well a model performs depends on the quality of its instructions, which makes it hard to evaluate an AI model."
 - **Instruction-following criteria:** IFEval and INFOBench measure the model's ability to follow a wide range of instructions, which are to give you ideas on how to evaluate a model's ability to follow your instructions.
 - **Roleplaying:** One of the most common types of real-world instructions is roleplaying — asking the model to assume the role of a fictional character or a

person. Roleplaying can serve two purposes:

1. Roleplaying a character for users to interact with, usually for entertainment, such as in gaming or interactive storytelling.
2. Roleplaying as a prompt engineering technique to improve the quality of a model's outputs.
 - Roleplaying capability evaluation is hard to automate. Benchmarks to evaluate roleplaying capability include **RoleLLM** and **CharacterEval**.
 - **CharacterEval** used human annotators and trained a reward model to evaluate each roleplaying aspect on a five-point scale.
 - **RoleLLM** evaluates a model's ability to emulate a persona using both carefully crafted similarity scores (How similar the generated outputs are to the expected outputs) and an AI judge.

System Instruction:

You are a role-playing performance comparison assistant. You should rank the models based on the role characteristics and text quality of their responses. The rankings are then output using Python dictionaries and lists.

User Prompt:

The models below are to play the role of “{role_name}”. The role description of “{role_name}” is “{role_description_and_catchphrases}”. I need to rank the following models based on the two criteria below:

1. Which one has more pronounced role speaking style, and speaks more in line with the role description. The more distinctive the speaking style, the better.
2. Which one’s output contains more knowledge and memories related to the role; the richer, the better. (If the question contains reference answers, then the role-specific knowledge and memories are based on the reference answer.)

- **Cost and Latency:** A model that generates high-quality outputs but is slow and expensive to run will not be useful. When evaluating models, it is important to balance model quality, latency, and cost. Many companies opt for lower-quality models if they provide better cost and latency.
 - Evaluation criteria for latency: time to first token, time per token, time between tokens, time per query, etc.
 - Latency depends not only on the underlying model but also on each prompt and sampling variable

Model Selection

"You need to think about which model is the best for your application."

During the application development process, as you progress through different adaptation techniques, you will have to do model selection over and over again. For example, prompt engineering might start with the strongest model overall to evaluate feasibility and then work backward to see if smaller models would work. If you decide to do fine-tuning, you might

start with a small model to test your code and move toward the biggest model that fits your hardware constraints (e.g., one GPU)

In general, the selection process for every technique consists of

1. Figuring out the best achievable performance
2. Mapping models along the cost-performance axes and choosing the model that gives the best performance for your budget.

Model Selection Workflow

- **Hard attributes** are often the results of decisions made by model providers (licenses, training data, model size) or your own policies (privacy, control). For some use cases, the hard attributes can reduce the pool of potential models significantly
- **Soft attributes** are attributes that can be improved upon, such as accuracy, toxicity, or factual consistency. When estimating how much you can improve on a certain attribute, it can be tricky to balance being optimistic and being realistic. I have had situations where a model's accuracy hovered around 20% for the first few prompts.

At a high level, the evaluation workflow consists of four steps:

- Filter out models whose hard attributes don't work for you, Your list of hard attributes depends heavily on your own internal policies, whether you want to use commercial APIs or host your own model.
- Use publicly available information, e.g., benchmark performance and leader board ranking, to narrow down the most promising models to experiment with, balancing different objectives such as model quality, latency, and cost
- Run experiment with your own evaluation pipeline to find the best model again, balancing all your objective.

Open Source Models

- A model should be considered open only if its training data is also made publicly available.

Open data allows more flexible model usage, such as reducing the model from scratch with modifications in the model architecture, training process, or the training data itself.

It also make it easier to understand the model:

- Sometimes, in some cases, we may require the model's training data for auditing purposes, e.g., to make sure that the model was not trained on compromised or illegally acquired data.

To signal whether the data is open,

- The term "**Open weight**" is used for model that don't come with open data,

- whereas the term **"Open model"** is used for models that come with open data.

The developers might hide some training data in purposes, as this information can open model developer to publicly scrutiny and potential lawsuit.

Before using any open sources, we have to evaluate several criteria including:

- Does the license allow commercial use?
- If it allows commercial use, are there any restrictions?
- Does the license allow using the model's output to train or improve upon other models?
in this case, the synthetic data as a solid example, which is generated by existing models, is an important source of data to train future models.

Model API vs Open Source Model

The service that hosts the model and receives user queries, runs the model to generate responses for queries, and return these response to the users is called **"An inference Service"**

The interface that the user interact with is called **"Model API"**

Typically, the term "Model API" refers to the API of the inference service, but there are also other APIs for other model's services, such as finetuning API, and evaluation APIs.

After developing a model, a developer can choose to open source it, making it accessible via an API, or both.

- Model API can be available through model providers (OpenAI, Anthropic,...)
- Cloud Service providers (Azure, GCP)
- Third party API providers (DataBrick Mosaic, Anyscale,...)

Whether to host a model yourself or use a model API depends on use case. So we have to consider: Data privacy, Data Lineage, Performance, Functionality, Costs, Control, and on-device deployment

Data privacy

Some companies may require strict policies, meaning that the data cannot be sent outside of the organization, if not this might exploit the company's policies.

For example: In Samsung case, the employees put company's secrets to the model. So Samsung banned the ChatGPT in May 2023.

To tackle this, the company comes up with an idea of prohibiting sending data outside their border.

And also, the model can also remember the training data by up to 8%. These memorized sample then can be accidentally leaked to users or intentionally exploited by bad actors.

Data Lineage and Copyright

So, in most model, there is no transparency whatsoever about what data it is trained on. Data Lineage is the process of investigating, understanding, recording, and visualizing the data as it flows from data source to consumption. This includes all transformation the data underwent along the way. What changed? and Why?

Performance

When it comes to performance, the model API tend to spark more fruitful impact as we do not have to care about the hardware cost, only the token computation cost is needed, while the self-hosting present the flexibility, in term of hardware, it might cost a lot, and sometime even 10 time bigger than the model API.

Functionality

Functionalities are needed to make the model work for a use case.

Functionalities example:

- **Scalability:** making sure the inference service can support your application's traffic while maintaining the desirable latency and cost
- **Function calling:** Giving the model the ability to use external tools, which is essential for RAG and agentic use cases
- **Structured outputs:** e.g., asking the model to output in JSON format
- **Output guardrails:** mitigating risks in the generated responses, such as making sure that the responses are not racist or sexist.

With model API, you are restricted to to the functionalities that the API provide.

+++ You can only finetune the commercial model unless it is allowed. If this model is proprietary and the model provider does not have a finetuning API, you will not be able to do it.

API cost versus engineering cost

- Commercial model provider charge per usage, which means they can get prohibitively expensive with heavy usage. So, at the certain scale, the company can consider host their own model in a certain scale.
- Using the model API means that you will have to depend on their SLA (Service-level agreement). So if these APIs are not reliable, you might need to spend a ton of time tinkering around with it.
- The model provides might not always give you the level of control you want. When using a service provider by someone, you are subjected to their terms and conditions, and their rate limits

	Using model APIs	Self-hosting models
Data	• <i>Have to send your data to model providers, which means your team can accidentally leak confidential info</i>	• Don't have to send your data externally • <i>Fewer checks and balances for data lineage/training data copyright</i>
Performance	• Best-performing model will likely be closed source	• <i>The best open source models will likely be a bit behind commercial models</i>
Functionality	• More likely to support scaling, function calling, structured outputs • <i>Less likely to expose logprobs</i>	• <i>No/limited support for function calling and structured outputs</i> • Can access logprobs and intermediate outputs, which are helpful for classification tasks, evaluation, and interpretability
Cost	• <i>API cost</i>	• <i>Talent, time, engineering effort to optimize, host, maintain (can be mitigated by using model hosting services)</i>
Finetuning	• <i>Can only finetune models that model providers let you</i>	• Can finetune, quantize, and optimize models (if their licenses allow), <i>but it can be hard to do so</i>

	Using model APIs	Self-hosting models
Control, access, and transparency	• <i>Rate limits</i> • <i>Risk of losing access to the model</i> • <i>Lack of transparency in model changes and versioning</i>	• Easier to inspect changes in open source models • You can freeze a model to maintain its access, <i>but you're responsible for building and maintaining model APIs</i>
Edge use cases	• <i>Can't run on device without internet access</i>	• Can run on device, <i>but again, might be hard to do so</i>

Navigating Public Benchmarks

Evaluation harness is a tool that helps you evaluate a model on multiple benchmarks.

Benchmark selection and aggregation

Benchmark results help you identify promising models for your use cases.

Aggregating benchmark results to rank model gives you a leaderboard.

Questions to consider:

- What benchmarks to include in your leaderboard?
- How to aggregate these benchmark results to rank models?

Public leaderboard: many public leaderboards rank models based on their aggregated performance on a subset of benchmarks.

Leaderboard / Benchmark	Measures	Best For
LMArena (formerly Chatbot Arena) ↗	Human preference voting via blind model comparisons	General chatbot quality
LiveBench ↗	Dynamic, contamination-resistant benchmark with fresh questions	Overall reasoning and real-world capability
SWE-bench Leaderboard ↗	Solving real GitHub software issues	Agentic coding
LiveCodeBench ↗	Fresh coding problems released after model training	Coding ability
Aider Leaderboard ↗	Code editing in real repositories	Practical software engineering
ARC Prize Leaderboard ↗	Abstract reasoning and general intelligence	AGI-style reasoning
GPQA Benchmark ↗	Graduate-level science reasoning	Expert knowledge reasoning
MMLU-Pro ↗	Knowledge and reasoning across many domains	Academic evaluation
WebArena Leaderboard ↗	Multi-step web browsing tasks	Web agents
OSWorld Leaderboard ↗	Operating a computer UI	Computer-use agents
Tau-Bench ↗	Tool use and enterprise workflows	Agent evaluation
AgentBench ↗	Multi-domain autonomous agent tasks	General agents

+++ For 2026 AI evaluation -- Source: ChatGPT

""Are OpenAI's Model is getting worse?""

User tend to complain that their models seem to be getting worse. And the evaluation significantly change between March 2023 and June 2023

One main potential reason is that the evaluation is hard, and no one, not even OpenAI, knows for sure if a model is getting worse or better.

Data Contamination with public benchmarks

Data Contamination = Data Leakage = training on the test set = cheating

All these terms are used interchangeably.

The Data Contamination happen when a model was trained on the same data it is evaluate on.

--> The model memorizes the answers it saw during the training, causing it to achieve higher evaluation scores than it should.

Cause:

- Some data is scraped from the internet, and the scrapping process can accidentally pull the data from various publicly available benchmark
- Can happen when the train and test data come from the same source.

However:

- Data contamination can also happen intentionally for good reasons. Let's say you want to create the best possible model for your users. Initially, you exclude benchmark data from the model's training data and choose the best model based on these benchmarks. However, because high-quality benchmark data can improve the model's performance, you then continue training your best model on benchmark data before releasing it to your users. So the released model is contaminated, and your users won't be able to evaluate it on contaminated benchmarks, but this might still be the right thing to do.

Handling:

Before handling the contamination, you have to detect the contamination, and then decontaminate your data. Detect it using:

- **N-gram overlapping:** If a sequence of 13 tokens in an evaluation sample is also in the training data, the model has likely seen this evaluation sample during training. This evaluation sample is considered dirty. (More accurate but time-consuming and expensive)
- **Perplexity:** Measure how difficult it is for a model to predict a given text. If a model's perplexity on evaluation data is usually low, meaning the model can easily predict the text, it is possible that the model has seen this data before during the training. (less accurate but less time-consuming and expensive)

Then:

- remove benchmarks they care about from their training data before training their models.

Ideally, when reporting the model's performance on a benchmark, it is helpful to disclose what percentage of this benchmark data is in your training data, and what the model's performance is on both the overall benchmark and the clean samples of the benchmark.

- Dealing with contamination (detecting and removing) is hard, some people find it easier to just skip it.

Paper: "Pretraining on the Test Set is All You Need" by Rylan Schaeffer

Idea: Training the model on data from several benchmarks, his 1m parameter model was able to achieve near-perfect scores and outperformed much larger models on all these benchmark.

Evaluation Pipeline

1. **Evaluate all components**

Evaluation can happen at different levels: Per task, Per turn, and Per intermediate output,

- **Turn-based evaluation:** evaluates the quality of each output.
- **Task-based evaluation:** evaluates whether a system complete a task. It is the most important, but the challenge is it can be hard to set the boundaries between the task

2. **Create the evaluation guideline**

- Define evaluation criteria
- Create scoring rubrics with examples
- Tie evaluation metrics to business metrics

3. **Select evaluation methods**

- Select evaluation methods
- Annotate evaluation data
- Evaluate the evaluation pipeline
- Iterate: as the needs and user behavior change, the evaluation criteria will also evolve and we may need to iterate on the evaluation pipeline.